

AIops Module 1 Assignment — Write-up

Q1 — Technical Debt Diagnosis (10 marks)

(a) Entanglement (CACE principle) — features share model parameters, so a change to one (rounding logic) shifts learned weights affecting unrelated features.

(b) Undeclared Consumers — marketing read the raw output table without the ML team's knowledge, creating an unmanaged dependency.

(c) Pipeline Jungles — 14 undocumented shell scripts with no orchestration is the classic glue-code/plumbing debt pattern.

Mitigation (for c): Replace the shell-script chain with a DAG-based orchestrator (e.g. Airflow/Dagster). Convert each script into a named task node with explicit declared inputs/outputs, let the orchestrator manage dependency ordering, retries and failure alerting, and add schema validation at stage boundaries. This turns an opaque linear chain into an inspectable, independently modifiable graph.

Q2 — MLflow Experiment Comparison (15 marks)

MLP on MNIST, 6 runs varying learning_rate x batch_size. MLflow run-comparison screenshot:

☐ Show diff only						
batch_size	128	16	128	16	128	16
hidden_layer_sizes	(128, 64)	(128, 64)	(128, 64)	(128, 64)	(128, 64)	(128, 64)
learning_rate	0.1	0.1	0.01	0.01	0.001	0.001
max_iter	60	60	60	60	60	60
model_type	MLPClassifier	MLPClassifier	MLPClassifier	MLPClassifier	MLPClassifier	MLPClassifier
Metrics						
☐ Show diff only						
final_train_loss	2.308	2.331	0.106	0.556	0.005	0.042
n_iter	60	20	30	12	28	29
train_loss	2.308	2.319	0.155	0.825	0.013	0.039
val_accuracy	0.113	0.099	0.948	0.809	0.974	0.97

Analysis: Best run is **run-2** (lr=0.001, batch=128): highest val_accuracy (97.4%), lowest train_loss (0.005). Overfitting evidence: runs 1,2,4 show train_loss near zero while val_accuracy plateaus lower — near-perfect training fit but slightly worse generalization. Runs 5-6 (lr=0.1) show optimization divergence, not overfitting — both train_loss and val_accuracy are poor simultaneously (~10% accuracy). **Learning rate has the larger effect:** lr=0.1 collapses accuracy to ~10% regardless of batch size; batch size alone shifts accuracy less dramatically (though at lr=0.01, batch=128 vs 16 is a large 80.9% to 94.8% jump).

```
mlflow.log_params({"learning_rate":cfg["lr"],"batch_size":cfg["batch_size"],"hidden_layer_sizes":cfg["hidden"]})
for epoch, loss_val in enumerate(clf.loss_curve_):
    mlflow.log_metric("train_loss", loss_val, step=epoch)
mlflow.log_metrics({"final_train_loss":final_train_loss,"val_accuracy":val_acc,"n_iter":clf.n_iter_})
```

Q3 — DVC Data Versioning & Rollback (10 marks)

Initialized Git + DVC in ml_project/; built file_list.csv (1800 rows + header). Configured an S3 remote and pushed v1 (dvc add data file_list.csv, git commit, git tag v1, dvc push). Simulated update by extracting new-labels.zip (1000 new images) into data/, rebuilt the CSV (2800 files + header = 2801 lines), re-ran dvc add, committed and tagged v2, and pushed. Rolled back with git checkout v1 followed by dvc checkout.

Terminal proof of rollback:

```
$ find data -type f|wc -l && wc -l file_list.csv -> 2800 / 2801 (v2)
$ git checkout v1 && dvc checkout
$ find data -type f|wc -l && wc -l file_list.csv -> 1800 / 1801 <- matches v1 exactly
```

Q4 — End-to-End Reproducibility Drill (15 marks)

Role: Partner B. Partner A trained a model, logged params/metrics/seed/git_commit tag/artifact to MLflow, versioned the dataset (Iris.csv) with DVC, committed code + .dvc file together, and registered/staged the model under experiment 'Reproducibility_Drill'.

Reproduction steps performed: git clone of A's repo, git checkout <commit>, dvc checkout (after resolving A's missing dvc remote config / push), mamba env create -f environment.yml, mamba activate, and re-ran question_4.ipynb headlessly via nbconvert. Compared the resulting run's logged metric against A's baseline run in the MLflow UI, then logged a verification note as a tag on my own MLflow run documenting the comparison result and tolerance, per the assignment's instructions.